

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Нейронные сети»
Тема: Многоклассовая классификация цветов

Студент гр. 1306

Шаврин А.П.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Реализовать классификацию сортов растения ирис (Iris Setosa - 0, Iris Versicolour - 1, Iris Virginica - 2) по четырем признакам: размерам пестиков и тычинок его цветков.

Задание.

- Ознакомиться с задачей классификации
- Загрузить данные
- Создать модель ИНС в Keras
- Настроить параметры обучения
- Обучить и оценить модель

Выполнение работы.

В качестве набора данных использовался классический датасет Iris. Данные были успешно загружены и содержат 150 образцов цветков ириса трех видов (Setosa, Versicolor, Virginica) (рисунок 1).

Предобработка данных включала в себя следующие шаги:

- **Разделение признаков и целевой переменной:** X (длина и ширина чашелистика и лепестка) и y (вид цветка).
- **Кодирование меток:** Целевая переменная, представленная строками, была преобразована в числовой формат с помощью LabelEncoder, а затем в формат one-hot encoding с помощью to_categorical. Это необходимо для использования функции потерь categorical_crossentropy.
- **Масштабирование признаков:** Все признаки были стандартизированы с помощью StandardScaler.
- **Разделение на выборки:** Данные были разделены на обучающую и тестовую выборки в соотношении 80/20. Для контроля качества обучения и настройки гиперпараметров 20% от обучающей выборки использовалось как валидационная выборка внутри процесса обучения.

```

  Чтение данных

[4]
✓ 0s
dataframe = pd.read_csv("Iris.csv", index_col=0)
print(dataframe.head())

  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  Species
Id
1          5.1           3.5           1.4           0.2  Iris-setosa
2          4.9           3.0           1.4           0.2  Iris-setosa
3          4.7           3.2           1.3           0.2  Iris-setosa
4          4.6           3.1           1.5           0.2  Iris-setosa
5          5.0           3.6           1.4           0.2  Iris-setosa

  Предобработка данных

[5]
✓ 0s
dataset = dataframe.values
X = dataset[:, 0:4].astype(float)
y = dataset[:, 4]

encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)
y_categorical = to_categorical(y_encoded)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_categorical,
    test_size=0.2,
    random_state=RANDOM_SEED,
    stratify=np.argmax(y_categorical, axis=1)
)

```

Рисунок 1. Чтение и предобработка данных.

Для проведения экспериментов была написана функция `create_model()` (рисунок 2), позволяющая задавать архитектуру сети через список `architecture`, где каждый элемент соответствует количеству нейронов в скрытом слое. Модель имеет входной слой, соответствующее количество скрытых слоев с активацией ReLU и выходной слой с 3 нейронами и функцией активации softmax для многоклассовой классификации.

```

  ▾ Генерация модели

[7] ✓ Os def create_model(architecture, learning_rate=0.001):
        """
        Создание модели с заданной архитектурой

        Параметры:
        architecture: список, где каждый элемент - количество нейронов в слое
        learning_rate: скорость обучения
        """
        model = Sequential()

        # Входной слой
        model.add(Dense(architecture[0], activation='relu', input_shape=(4,)))

        # Скрытые слои
        for neurons in architecture[1:]:
            model.add(Dense(neurons, activation='relu'))

        # Выходной слой
        model.add(Dense(3, activation='softmax'))

        # Компиляция
        optimizer = Adam(learning_rate=learning_rate)
        model.compile(
            optimizer=optimizer,
            loss='categorical_crossentropy',
            metrics=['accuracy']
        )

        return model

```

Рисунок 2. Функция создания модели.

1. Сравнение архитектур нейронной сети

В этом эксперименте было обучено 8 моделей с различной архитектурой, от простой ([4]) до более сложных ([32, 16, 8]). Обучение каждой модели проводилось в течение 100 эпох с размером батча равном 16. На рисунках 3 и 4 представлены графики точности и потерь для каждой из архитектур.

Графики точности для всех архитектур

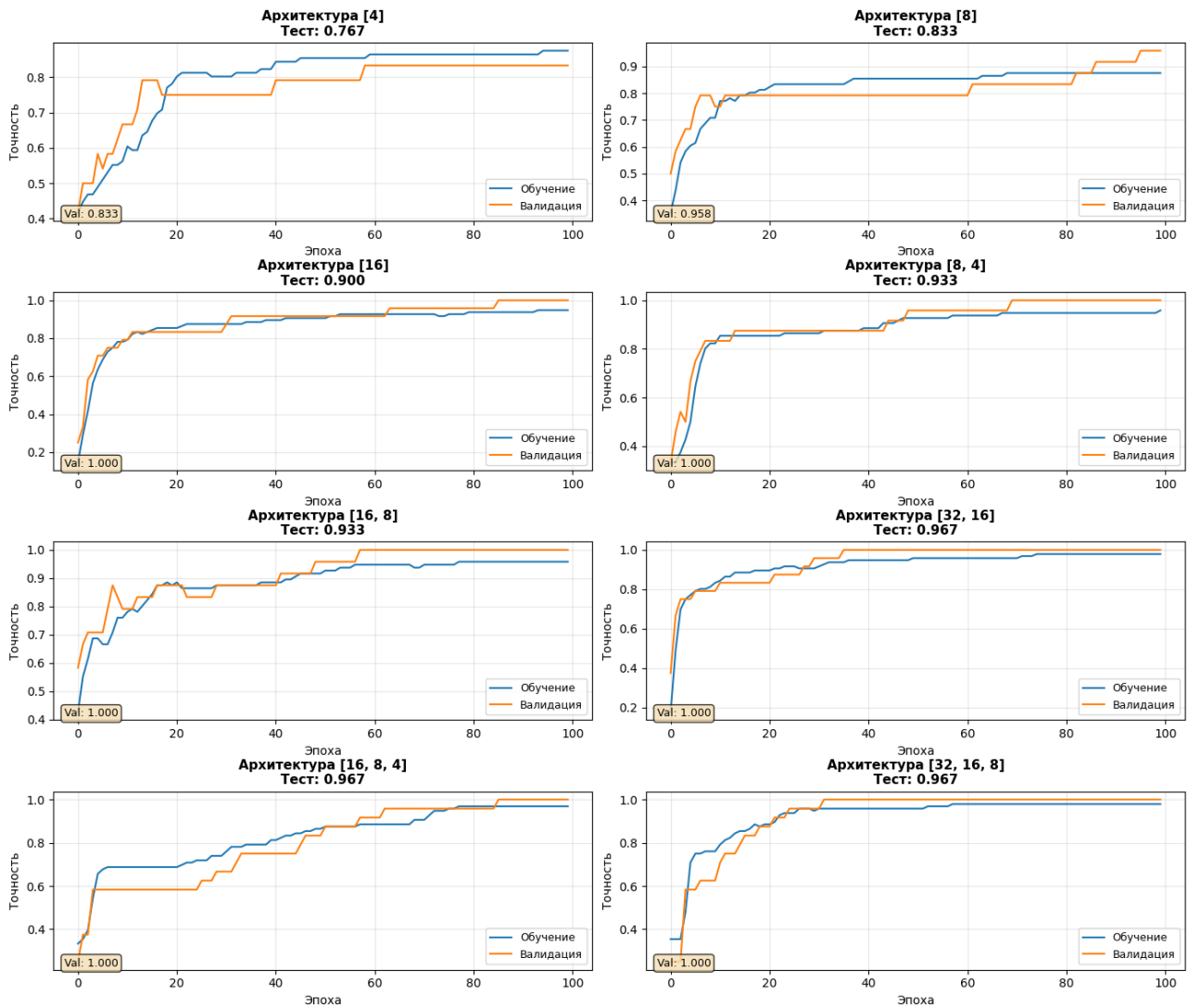


Рисунок 3. Графики точности для всех архитектур.

Графики потерь для всех архитектур

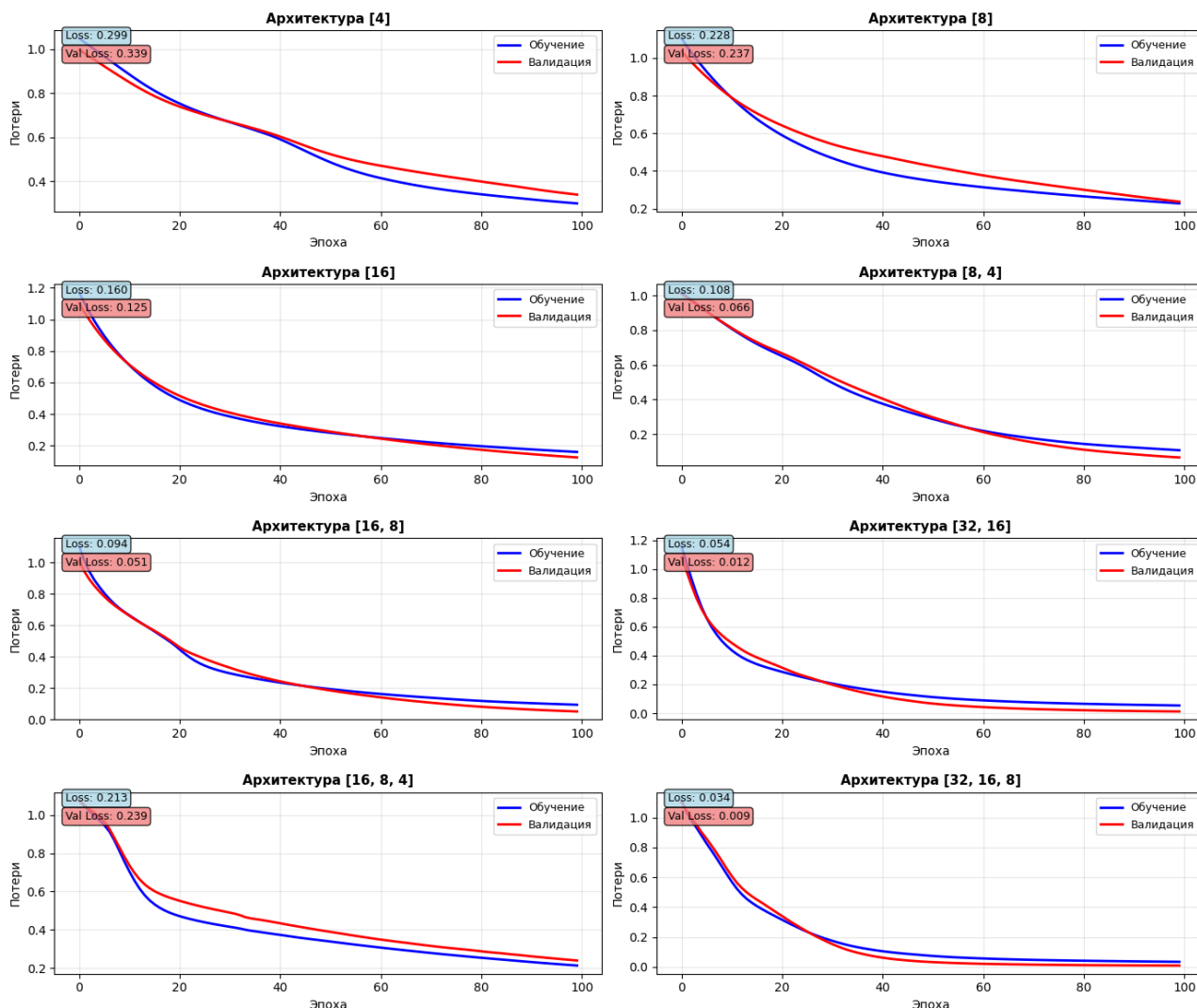


Рисунок 4. Графики потерь для всех архитектур.

Исходя из графиков можно сделать выводы:

- Базовая архитектура [4] показала наименьшую точность на тестовой выборке (0.7667). Это ожидаемо, так как модель с 4 нейронами имеет ограниченную способность к обучению.
- Усложнение архитектуры (увеличение количества нейронов и слоев) привело к значительному росту точности. Уже модель [8] дала прирост, а архитектуры с 16 нейронами и более достигли точности 0.9667 на тестовой выборке.
- Начиная с архитектуры [16], дальнейшее усложнение ([32, 16], [16, 8, 4], [32, 16, 8]) не привело к улучшению тестовой точности, которая

стабилизировалась на уровне 0.9667. Это говорит о том, что для данной задачи такой сложности модели достаточно.

Анализируя графики обучения также можно заметить следующее:

- Графики показывают, что модели с малой емкостью (например, [4]) не могут достичь высоких значений точности на обучающей выборке. Более сложные модели ([16] и выше) быстро достигают 100% точности на обучении, что свидетельствует о том, что они способны выучить данные.
- Графики потерь коррелируют с точностью. Для более сложных моделей потери на обучении стремятся к нулю.
- Переобучение отсутствует: Видно, что даже при 100% точности на обучении, модели с архитектурами [16] и выше демонстрируют очень высокую точность на валидации и тесте. Разрыв между кривыми обучения и валидации минимален, что говорит об отсутствии серьезного переобучения, несмотря на способность модели идеально запомнить обучающие данные. Это связано с простотой и линейной разделимостью данных Iris.

Таким образом, можно сделать вывод, что для задачи классификации ирисов оптимальными являются архитектуры с одним-двумя скрытыми слоями и 16 и более нейронами. Дальнейшее усложнение модели не дает выигрыша в точности. Архитектуры [16], [16, 8], [32, 16], [16, 8, 4] и [32, 16, 8] показали наилучший результат.

2. Влияние различных параметров обучения

В качестве базовой архитектуры для последующих экспериментов с гиперпараметрами была выбрана простейшая однослойная сеть с 4 нейронами ([4]). Это решение позволяет наглядно продемонстрировать, как изменение гиперпараметров может улучшить или ухудшить производительность даже самой простой модели.

Влияние размера батча (Batch Size)

Исследовалось влияние размера батча на обучение базовой модели. Результаты представлены на рисунке 5 и 6.

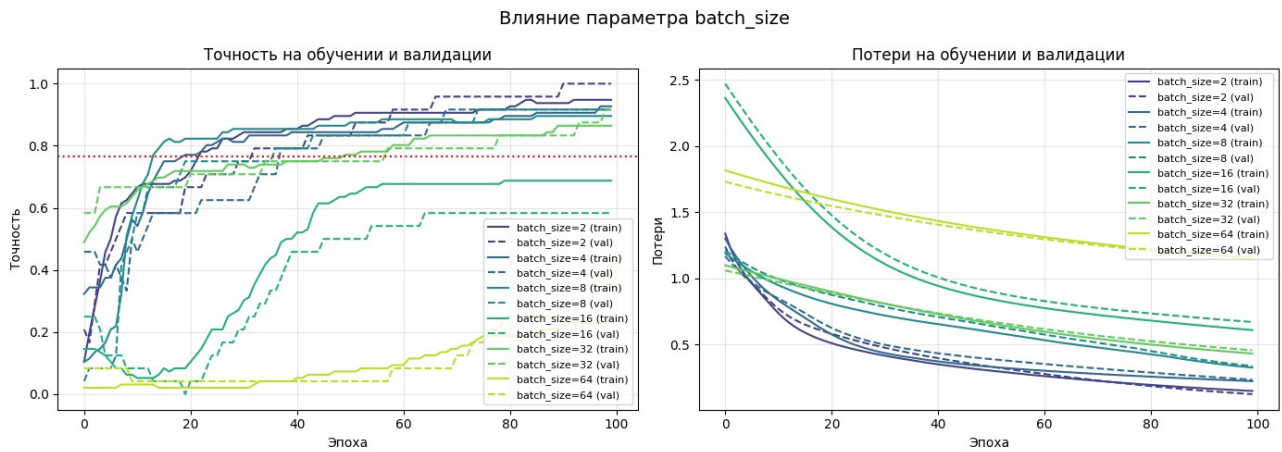


Рисунок 5. Графики точности и потерь для различных размеров батча.

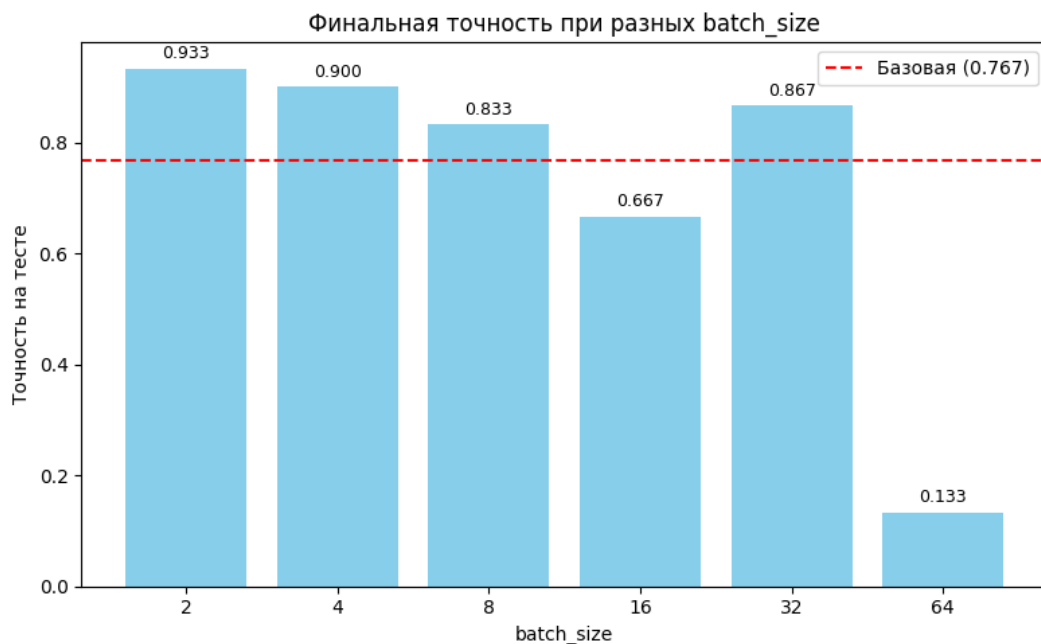


Рисунок 6. Графики финальной точности для различных размеров батча.

Исходя из результатов экспериментов можно заметить:

- Малые батчи (2, 4): Позволили модели достичь наивысшей точности (0.9333 для batch_size=2). Более высокая частота обновления весов способствовала более эффективному исследованию функции потерь.
- Средние батчи (8, 16, 32): Показали варьирующиеся результаты. batch_size=8 дал результат 0.8333, batch_size=32 - 0.8667, что выше базового, но ниже, чем при малых батчах.
- Большой батч (64): Привел к падению точности (0.1333). Вероятная причина - использование чрезмерно большого размера батча, что

вызывает сглаживание градиента и преждевременную сходимость к неоптимальным областям.

Таким образом для данной задачи наиболее эффективным оказался малый размер батча (2), позволивший добиться наилучшей точности.

Влияние количества эпох (Epochs)

Цель эксперимента - определить, сколько итераций обучения необходимо для достижения наилучшего результата и как долго можно обучать модель без переобучения. Результаты представлены на рисунке 7 и 8.

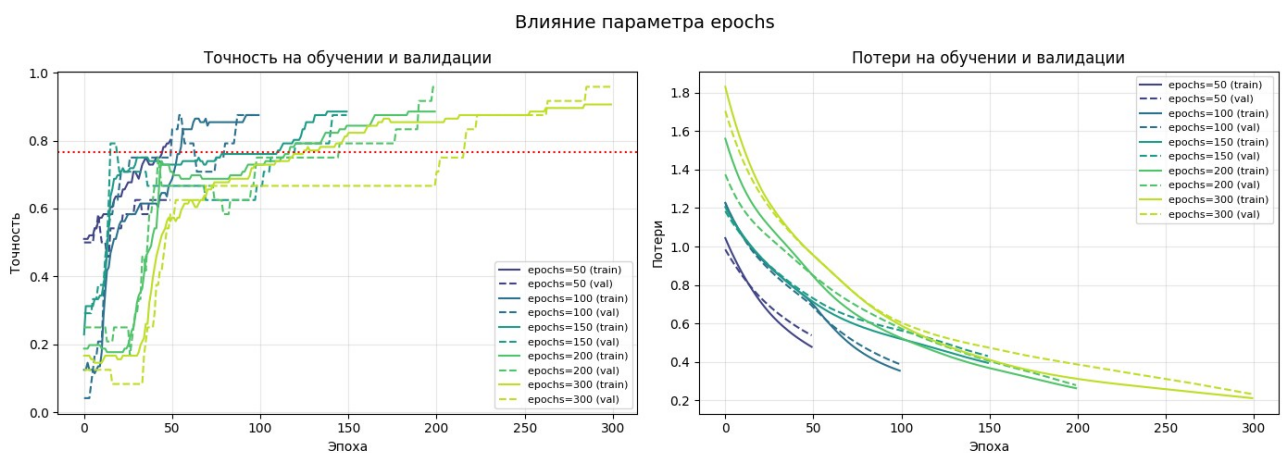


Рисунок 7. Графики точности и потерь для различного числа эпох.

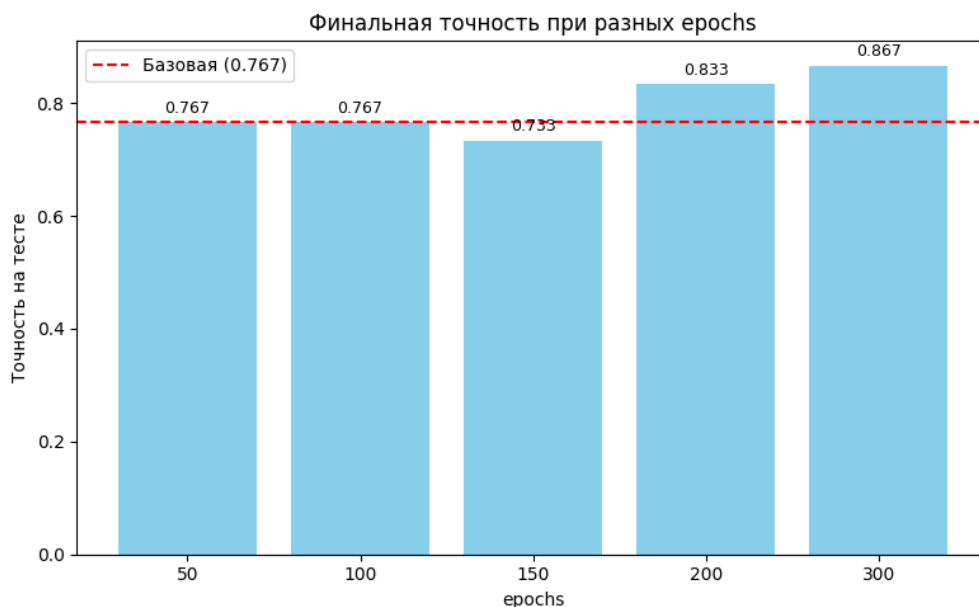


Рисунок 8. Графики финальной точности и потерь для различного числа эпох.

Исходя из результатов экспериментов можно заметить:

- 50 эпох: Модель показала результат, идентичный базовому (0.7667), но с большим разрывом между точностью на обучении и валидации (0.1771), что указывает на незавершенность процесса обучения.
- 100-150 эпох: При 100 эпохах результат не изменился, а при 150 - даже упал (0.7333). Это говорит о нестабильности базовой модели.
- 200 и 300 эпох: Дальнейшее увеличение числа эпох привело к значительному росту точности (до 0.8333 и 0.8667). Это может быть связано с тем, что базовая модель [4] очень проста.

Таким образом для базовой модели увеличение числа эпох до 300 дало наилучший результат. Однако, для более сложных архитектур (из первого эксперимента), 100 эпох было достаточно для достижения пика производительности. Это подчеркивает, что оптимальное число эпох зависит от сложности модели и данных.

Влияние доли валидационной выборки (Validation Split)

Этот эксперимент проверял, как размер данных, отведенных для валидации, влияет на итоговую точность модели на тесте. Результаты представлены на рисунке 9 и 10.

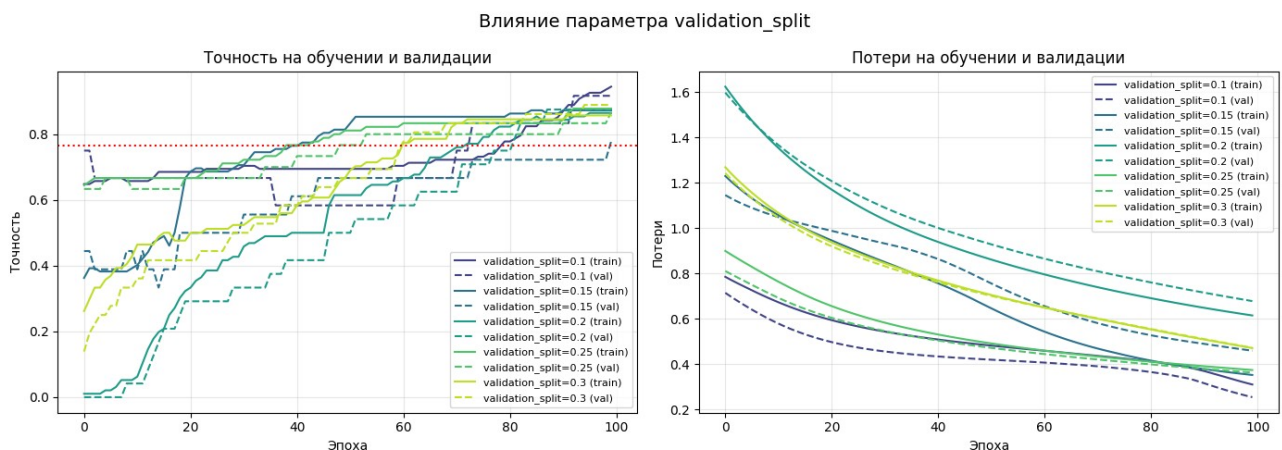


Рисунок 9. Графики точности и потерь для различного размера валидационной выборки.

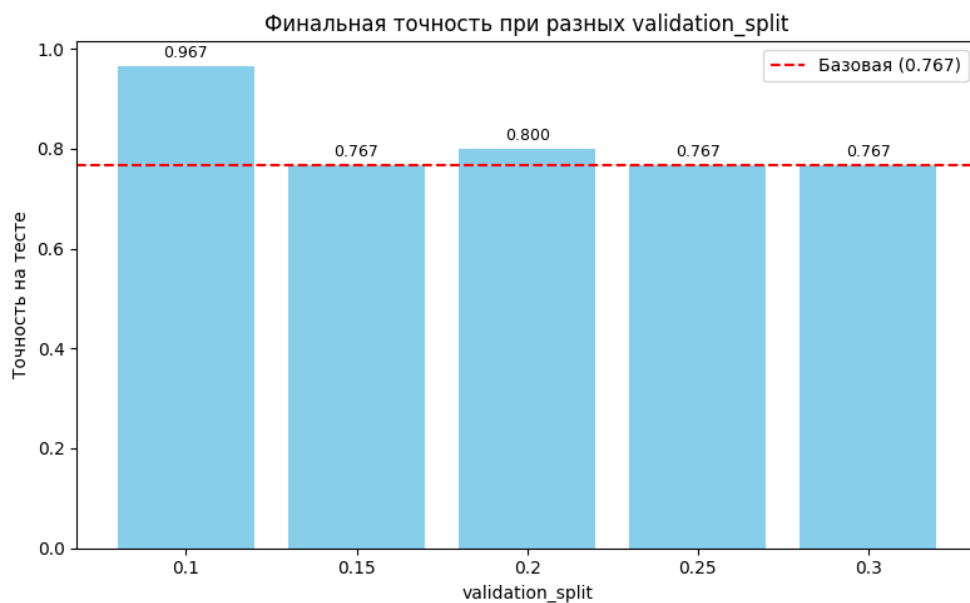


Рисунок 10. Графики финальной точности и потерь для различного размера валидационной выборки.

Исходя из результатов экспериментов можно заметить:

- Наилучший результат (0.9667) был достигнут с самой маленькой долей валидации - `validation_split=0.1`. Это означает, что модель видела больше данных для обучения (90% от тренировочной выборки), что для небольшого датасета Iris оказалось решающим фактором для достижения наилучшего обобщения.
- Другие значения (0.15, 0.2, 0.25, 0.3) показали результаты, близкие к базовому или немного выше, но уступили варианту с наименьшей валидационной выборкой.

Таким образом для небольших датасетов использование минимально возможной валидационной выборки может быть предпочтительным, так как позволяет использовать максимум данных для обучения.

Выводы.

В ходе лабораторной работы была реализована классификацию сортов растения ирис по четырем признакам: размерам пестиков и тычинок его цветков. Проведенные эксперименты позволяют сделать следующие выводы:

Архитектура сети имеет решающее значение. Простые модели ([4]) имеют недостаточную емкость, в то время как архитектуры с 16 и более нейронами ([16], [16, 8], [32, 16], [16, 8, 4], [32, 16, 8]) способны достичь точности 96.67% на тестовой выборке. Дальнейшее усложнение архитектуры не приводит к улучшению результата на данных Iris.

Гиперпараметры обучения также критически важны. Даже для простой модели ([4]) правильный подбор гиперпараметров позволяет значительно повысить точность:

- Лучший размер батча: 2 (точность 93.33%).
- Лучшее количество эпох: 300 (точность 86.67%).
- Лучшая доля валидационной выборки: 0.1 (точность 96.67%).

Лучшая модель по итогам всех экспериментов:

Учитывая все проведенные исследования, наилучшей конфигурацией, позволившей достичь максимальной точности в 96.67%, является одна из архитектур, показавших этот результат в первом эксперименте (например, [16], [16, 8] или др.). Выбор конкретной архитектуры из этой группы может быть сделан в пользу самой простой из них ([16]), так как она требует меньше вычислительных ресурсов и не уступает в точности.

В качестве оптимального гиперпараметра для обучения, если использовать модель [16], можно рекомендовать `batch_size=2` (или 4), `epochs=100` и `validation_split=0.1`. Эксперименты показали, что для достижения максимального качества критически важно не только выбрать правильную архитектуру, но и точно настроить параметры процесса обучения, особенно размер батча и объем данных, используемых для валидации.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lab1.ipynb

```
# Установка зависимостей
"""
```

```
!pip install pandas numpy matplotlib scikit-learn tensorflow
```

```
""""# Импорт""""
```

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import random
import tensorflow as tf
from sklearn.model_selection import train_test_split
```

```
RANDOM_SEED = 42
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)
tf.random.set_seed(RANDOM_SEED)
tf.config.experimental.enable_op_determinism()
```

```
""""# Чтение данных""""
```

```
dataframe = pd.read_csv("Iris.csv", index_col=0)
print(dataframe.head())
```

```
""""# Предобработка данных""""
```

```
dataset = dataframe.values
X = dataset[:, 0:4].astype(float)
y = dataset[:, 4]
```

```
encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)
y_categorical = to_categorical(y_encoded)
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_categorical,
    test_size=0.2,
    random_state=RANDOM_SEED,
```

```

    stratify=np.argmax(y_categorical, axis=1)
)

"""# Генерация модели"""

def create_model(architecture, learning_rate=0.001):
    """
    Создание модели с заданной архитектурой

    Параметры:
    architecture: список, где каждый элемент - количество нейронов в слое
    learning_rate: скорость обучения
    """
    model = Sequential()

    # Входной слой
    model.add(Dense(architecture[0], activation='relu', input_shape=(4,)))

    # Скрытые слои
    for neurons in architecture[1:]:
        model.add(Dense(neurons, activation='relu'))

    # Выходной слой
    model.add(Dense(3, activation='softmax'))

    # Компиляция
    optimizer = Adam(learning_rate=learning_rate)
    model.compile(
        optimizer=optimizer,
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

"""# Сравнение Архитектур"""

architectures = [
    [4],
    [8],
    [16],
    [8, 4],
    [16, 8],
    [32, 16],
    [16, 8, 4],
    [32, 16, 8],
]
architecture_results = []

for arch in architectures:
    print(f"\nОбучение архитектуры: {arch}")

```

```

model = create_model(arch)

history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=10,
    validation_split=0.2,
    verbose=0
)

# Оценка на тестовой выборке
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)

architecture_results.append({
    'architecture': arch,
    'final_train_acc': history.history['accuracy'][-1],
    'final_val_acc': history.history['val_accuracy'][-1],
    'test_acc': test_acc,
    'history': history
})

print(f"\tТочность на валидации: {history.history['val_accuracy'][-1]:.4f}")
print(f"\tТочность на тесте: {test_acc:.4f}")

def plot_accuracy_grid(architecture_results, architectures):
    """
    Построение сетки 4x2 графиков точности для всех архитектур
    """
    fig, axes = plt.subplots(4, 2, figsize=(14, 12))
    fig.suptitle('Графики точности для всех архитектур', fontsize=14, y=1.02)

    for idx, (arch, ax) in enumerate(zip(architectures, axes.flat)):
        result = next(r for r in architecture_results if r['architecture'] == arch)
        history = result['history']

        ax.plot(history.history['accuracy'], label='Обучение')
        ax.plot(history.history['val_accuracy'], label='Валидация')

        ax.set_xlabel('Эпоха', fontsize=10)
        ax.set_ylabel('Точность', fontsize=10)
        ax.set_title(f'Архитектура {arch}\nТест: {result["test_acc"]:.3f}',
                     fontsize=11, fontweight='bold')
        ax.legend(loc='lower right', fontsize=9)
        ax.grid(True, alpha=0.3)

        ax.text(0.02, 0.02, f'Val: {result["final_val_acc"]:.3f}',
               transform=ax.transAxes, fontsize=9,
               bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

    plt.tight_layout()
    plt.subplots_adjust(top=0.95, hspace=0.4, wspace=0.1)
    plt.savefig('accuracy_grid.png', dpi=300, bbox_inches='tight')

```

```
plt.show()
```

```
def plot_loss_grid(architecture_results, architectures):
    """
    Построение сетки 4x2 графиков потерь для всех архитектур
    """
    fig, axes = plt.subplots(4, 2, figsize=(14, 12))
    fig.suptitle('Графики потерь для всех архитектур', fontsize=14, y=1.02)

    for idx, (arch, ax) in enumerate(zip(architectures, axes.flat)):
        result = next(r for r in architecture_results if r['architecture'] == arch)
        history = result['history']

        ax.plot(history.history['loss'], 'b-', linewidth=2, label='Обучение')
        ax.plot(history.history['val_loss'], 'r-', linewidth=2, label='Валидация')

        ax.set_xlabel('Эпоха', fontsize=10)
        ax.set_ylabel('Потери', fontsize=10)
        ax.set_title(f'Архитектура {arch}', fontsize=11, fontweight='bold')
        ax.legend(loc='upper right', fontsize=9)
        ax.grid(True, alpha=0.3)

        ax.text(0.02, 0.95, f'Loss: {history.history["loss"][-1]:.3f}',
                transform=ax.transAxes, fontsize=9,
                bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.8))
        ax.text(0.02, 0.85, f'Val Loss: {history.history["val_loss"][-1]:.3f}',
                transform=ax.transAxes, fontsize=9,
                bbox=dict(boxstyle='round', facecolor='lightcoral', alpha=0.8))

    plt.tight_layout()
    plt.subplots_adjust(top=0.95, hspace=0.4, wspace=0.1)
    plt.savefig('loss_grid.png', dpi=300, bbox_inches='tight')
    plt.show()
```

```
plot_accuracy_grid(architecture_results, architectures)
plot_loss_grid(architecture_results, architectures)
```

```
"""# Сравнение параметров обучения"""
```

```
def plot_parameter_comparison(results, param_name, param_values, baseline_acc=None,
metric='accuracy'):
    """
    Строит два подграфика: слева – точность, справа – потери.
    Для каждого значения параметра рисуются кривые обучения и валидации.
    """
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
    fig.suptitle(f'Влияние параметра {param_name}', fontsize=14)

    colors = plt.cm.viridis(np.linspace(0.2, 0.9, len(results)))
```



```

for res, color in zip(results, colors):
    val = res['value']
    hist = res['history']
    # Точность
    ax1.plot(hist.history['accuracy'], color=color, linestyle='-',
             label=f'{param_name}={val} (train)')
    ax1.plot(hist.history['val_accuracy'], color=color, linestyle='--',
             label=f'{param_name}={val} (val)')
    # Потери
    ax2.plot(hist.history['loss'], color=color, linestyle='-',
             label=f'{param_name}={val} (train)')
    ax2.plot(hist.history['val_loss'], color=color, linestyle='--',
             label=f'{param_name}={val} (val)')

ax1.set_xlabel('Эпоха')
ax1.set_ylabel('Точность')
ax1.set_title('Точность на обучении и валидации')
ax1.legend(loc='lower right', fontsize=8)
ax1.grid(True, alpha=0.3)
if baseline_acc is not None:
    ax1.axhline(y=baseline_acc, color='red', linestyle='-',
               label=f'Базовая модель ({baseline_acc:.3f})')

ax2.set_xlabel('Эпоха')
ax2.set_ylabel('Потери')
ax2.set_title('Потери на обучении и валидации')
ax2.legend(loc='upper right', fontsize=8)
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

def plot_final_accuracy(results, param_name, baseline_acc):
    values = [r['value'] for r in results]
    test_accs = [r['test_acc'] for r in results]

    plt.figure(figsize=(8, 5))
    bars = plt.bar(range(len(values)), test_accs, tick_label=values, color='skyblue')
    plt.axhline(y=baseline_acc, color='red', linestyle='--', label=f'Базовая
({baseline_acc:.3f})')
    plt.xlabel(param_name)
    plt.ylabel('Точность на тесте')
    plt.title(f'Финальная точность при разных {param_name}')
    plt.legend()
    # Подписи значений над столбцами
    for bar, acc in zip(bars, test_accs):
        plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                 f'{acc:.3f}', ha='center', va='bottom', fontsize=9)
    plt.tight_layout()
    plt.show()

"""## Размер батча"""

```

```

TARGET_ARCH = architectures[0]
BASELINE_ACC = architecture_results[0]['test_acc']
batch_results = []
batch_sizes = [2, 4, 8, 16, 32, 64]
batch_best_acc = 0
batch_best_value = None

for batch_size in batch_sizes:
    print(f"\n* batch_size = {batch_size}")

    model = create_model(TARGET_ARCH)

    history = model.fit(
        X_train, y_train,
        epochs=100,
        batch_size=batch_size,
        validation_split=0.2,
        verbose=0
    )

    test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)

    print(f"\tТочность на тесте: {test_acc:.4f} (было: {BASELINE_ACC:.4f})")
    print(f"\tИзменение: {test_acc - BASELINE_ACC:+.4f}")

    if test_acc > batch_best_acc:
        batch_best_acc = test_acc
        batch_best_value = batch_size

    batch_results.append({
        'value': batch_size,
        'test_acc': test_acc,
        'history': history
    })

print(f"\nЛучший batch_size: {batch_best_value} (точность: {batch_best_acc:.4f})")

baseline = architecture_results[0]['test_acc']
plot_parameter_comparison(batch_results, 'batch_size', batch_sizes, baseline)

"""## Количество эпох"""

TARGET_ARCH = architectures[0]
BASELINE_ACC = architecture_results[0]['test_acc']
epochs_results = []
epochs_list = [50, 100, 150, 200, 300]
epochs_best_acc = 0
epochs_best_value = None

for epochs in epochs_list:
    print(f"\n* epochs = {epochs}")

```

```

model = create_model(TARGET_ARCH)

history = model.fit(
    X_train, y_train,
    epochs=epochs,
    batch_size=16,
    validation_split=0.2,
    verbose=0
)

test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)

# Анализ переобучения
train_acc = history.history['accuracy'][-1]
val_acc = history.history['val_accuracy'][-1]
overfitting = train_acc - val_acc

print(f"\tТочность на тесте: {test_acc:.4f} (было: {BASELINE_ACC:.4f})")
print(f"\tИзменение: {test_acc - BASELINE_ACC:+.4f}")
print(f"\tРазрыв train-val: {overfitting:.4f}")

if test_acc > epochs_best_acc:
    epochs_best_acc = test_acc
    epochs_best_value = epochs

epochs_results.append({
    'value': epochs,
    'test_acc': test_acc,
    'history': history
})

print(f"\nЛучшее количество эпох: {epochs_best_value} (точность:
{epochs_best_acc:.4f})")

baseline = architecture_results[0]['test_acc']
plot_parameter_comparison(epochs_results, 'epochs', epochs_list, baseline)

"""## Соотношение разбиения данных для валидации"""

TARGET_ARCH = architectures[0]
BASELINE_ACC = architecture_results[0]['test_acc']
split_results = []
val_splits = [0.1, 0.15, 0.2, 0.25, 0.3]
split_best_acc = 0
split_best_value = None

for val_split in val_splits:
    print(f"\n* validation_split = {val_split}")

    model = create_model(TARGET_ARCH)

```

```

history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=16,
    validation_split=val_split,
    verbose=0
)

test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)

print(f"\tТочность на тесте: {test_acc:.4f} (было: {BASELINE_ACC:.4f})")
print(f"\tИзменение: {test_acc - BASELINE_ACC:+.4f}")

if test_acc > split_best_acc:
    split_best_acc = test_acc
    split_best_value = val_split

split_results.append({
    'value': val_split,
    'test_acc': test_acc,
    'history': history
})

print(f"\nЛучший validation_split: {split_best_value} (точность: {split_best_acc:.4f})")

baseline = architecture_results[0]['test_acc']
plot_parameter_comparison(split_results, 'validation_split', val_splits, baseline)

baseline = architecture_results[0]['test_acc']
plot_final_accuracy(batch_results, 'batch_size', baseline)
plot_final_accuracy(epochs_results, 'epochs', baseline)
plot_final_accuracy(split_results, 'validation_split', baseline)

```